

What a fast decision model buys an AI agent

Telemetry, benchmarks and paired sessions, from curiosity to
configuration

Amplifier

Michael J. Jabbour

David Koleczek
Senior Applied Scientist

Microsoft, Office of the CTO

October 6, 2026 · report v3

An AI agent makes many small decisions: which model should run this session, may this file be read without asking the expensive model, how hard should the model think. A “system 1” decision model answers such typed questions in a fraction of a second. We asked what one buys an agent harness in quality, cost and speed. Live telemetry (464,524 events from 2026-09-17 to the 2026-10-05 cutoff, 171,609 of them in the first week) showed decisions cost 159 ms against 4.8 s for a host model call, but that the headline agreement figure mostly measured a labelling artefact. A preregistered judge benchmark found Jev, Typesafe’s decision model, the most accurate per dollar; no judge was useful on real read decisions. A paired campaign of 1,036 multi-turn sessions measured what routing a session to a cheaper model really costs: with Fable 5.1 as host, routing once per session to Sonnet cost $0.56\times$ the plain host (geometric mean of per-pair ratios); with Opus 5.5 as host every routing arm cost more. Two preregistered follow-ups showed that part of that saving is reasoning effort (Sonnet at medium $0.821\times$ its default; Fable at medium $0.860\times$). A counterfactual analysis then suggested, and a preregistered holdout study of 1,224 sessions confirmed, that a one-line rule matches the decision model at the session-start choice ($1.008\times$, equivalent). On fresh scenarios the frozen Fable configuration cost $0.581\times$ plain Fable; routing on Opus still lost ($1.255\times$); medium effort on Opus did not demonstrate a saving. We end with what to configure, per host model and per harness, and with what we have not yet measured.

Technical report. Every number in this document is generated from committed evidence files by `build_assets.py`; see [Section A](#).

The short version

The question. A coding agent spends its money on calls to a large *host model*. A fast “system 1” *decision model* can answer small typed questions in milliseconds: which model should run this session, may this file be read without asking, how hard should the model think. We asked what such a model buys an agent harness in quality, cost and speed, and followed the answer through telemetry, two judge benchmarks, a paired campaign of whole sessions and a preregistered holdout study (S1).

What we found.

1. **Decisions are cheap and fast; telemetry cannot say whether they are right.** A Jev decision took 159 ms at the median against 4.8 s for a host call, but the recorded “agreement” mostly measured a labelling artefact (Section 2).
2. **On labelled decisions Jev is the best value; on real read decisions no judge is safe enough.** No accuracy difference from dearer judges was detected after correction, and 0 of 10 judges met the usefulness rule on real reads (Section 3).
3. **Routing a whole session to Sonnet pays on Fable and costs on Opus.** On fresh scenarios the frozen Fable configuration cost $0.581\times$ plain Fable (95 % CI 0.542–0.625); routing on Opus cost $1.255\times$ (1.197–1.321), because Opus’s cache reads are cheap (Sections 4 and 5).
4. **At session start, a one-line rule matches the decision model.** The rule and Jev were equivalent ($1.008\times$, 90 % CI 1.000–1.019) and disagreed on only 4 of 120 scenario-reps (Section 5).
5. **Reasoning effort is a second lever, but not on every host.** Medium effort cost $0.821\times$ the default on Sonnet and $0.860\times$ on Fable; on Opus it did not demonstrate a saving ($0.992\times$).

What to do on Monday. Decide once per session. Let the price gate decide whether routing can pay: never on Opus, to Sonnet at medium effort on Fable. Use the rule rather than the decision model for that choice. Keep review and explain work on the host, where non-inferiority was not established; with the shipped keyword proxy that costs about $0.75\times$ of plain Fable instead of about $0.58\times$. The exact settings are in Sections 7 and 8.

The one-sentence answer. A fast decision model is worth what its decisions change: at the start of a session, price arithmetic and a one-line rule captured all of the measured saving, so the place where such a model might still earn its keep is in decisions whose right answer changes during a session, which we have not yet measured.

Contents

The short version	1
1 What does a fast decision model buy an agent?	3
2 Telemetry shows that decisions are fast, not whether they are right	5
3 On labelled decisions Jev is the best value; on real reads no judge is safe	6
4 In whole sessions, routing pays on Fable and costs on Opus	11
5 At session start, a one-line rule matches the decision model	13
6 Inside S1: where the result is solid and where it is thin	16
7 What to configure on Monday	22
8 Configuration recipes	24
9 What this means, and what remains open	26
A Companion material	27
B Freeze-rule configurations	27
C Glossary	29
D Response to the independent review (round 3)	30

1 What does a fast decision model buy an agent?

Coding agents spend money in one place: calls to a large *host model*. Every turn re-sends the conversation, and a session of a dozen turns can cost several dollars. Between those calls sit many small choices. Should this session run on the expensive model or a cheaper one? May the agent read a file without asking the model first? How much should the model “think”? A *decision model* (“system 1”) answers such questions: it takes a short state and a typed question and returns a probability for each allowed answer, in tens to hundreds of milliseconds. Jev, by Typesafe, is one; GPT-6 Luna and Sol, Cloudflare’s Clef and Clef-Flash, and small local models (Qwen3, Laya) can play the same role.

To keep the numbers concrete we follow one real task through the report: *click-flow-explain*, an explain task from the S1 holdout. On plain Fable one of its sessions cost \$4.04 on average; that is the scale of money a single session-start decision moves.

Our question was simple, and we followed it where it went: *what does a decision model buy an agent harness in quality, cost and speed?*

Key idea. A decision model is only worth what its decisions change. It pays off when a cheap, typed decision changes an expensive action, and it adds nothing when a fixed rule makes the same choice. Most of this report is about telling those two cases apart with measurements rather than estimates.

How the story goes.. We first looked at the bundle’s live telemetry (Section 2). We then built tools to grade decision models against labelled cases and real traces (Section 3), raised the difficulty to full paired agent sessions (Section 4), and reached a plateau where the question became how to configure the model rather than which one to pick (Section 5). Section 7 says what to set, per host model and per harness; Section 9 says what we did not finish.

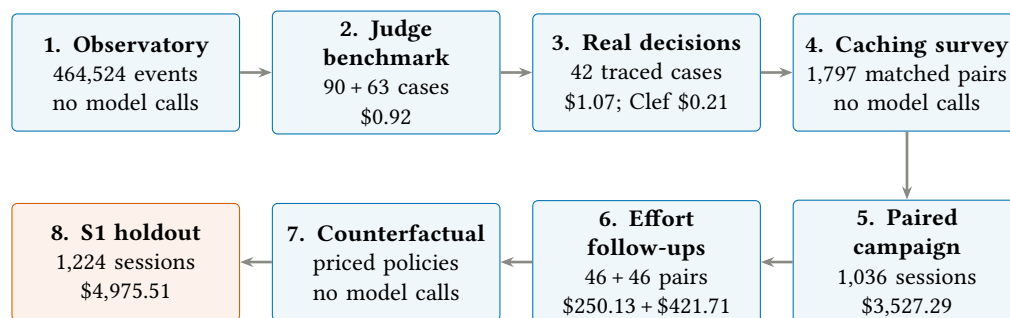


Figure 1: Each stage answered the question the previous one left open. The study program in order: what each stage measured, how big it was and what it cost in model calls.

How to read it. Follow the arrows. Each stage answered a question the previous one raised: telemetry could not say whether decisions were right (so we built benchmarks), benchmarks could not say what a decision saves (so we ran whole sessions), and whole sessions raised the effort and decider questions that the follow-ups and S1 settled.

Related work.. Routing work to cheaper models is not new. RouteLLM learns, query by query, when a weaker model’s answer will be good enough (<https://arxiv.org/abs/2406.18665>); FrugalGPT cascades queries through models of increasing cost and stops when an answer is judged reliable (<https://arxiv.org/abs/2305.05176>). Both treat a query as one call. This report measures something they do not: whole multi-turn agent sessions, where prompt caching

makes the price of switching models depend on the conversation so far, where reasoning effort is a second cost lever, and where a learned decider is tested against a deterministic rule rather than against no routing at all. We did not conduct an exhaustive literature search.

How to read the evidence labels.. Results marked [\[preregistered\]](#) test hypotheses fixed before the data; [\[screen\]](#) marks exploratory results. Cost ratios are *geometric means* of per-pair ratios unless labelled otherwise, and a 95 % interval is given wherever one exists. Every number is generated from committed evidence files. The full detail of each study, with its own figures and its peer review, is in the companion technical report v2 (docs/papers/2026-10-02-paired-measurement/), cited here as “the companion report”.

Table 1: Findings at a glance. “Preregistered” means the hypothesis and its threshold were fixed before the data; “replicated” means an independent set of scenarios gave the same answer; “exploratory” findings are hypotheses.

Finding	Number (95 % CI unless noted)	Study	Evidence
Decision calls are fast next to host calls	Jev p50 159 ms; host p50 4.8 s	observatory	measured
Shadow “agreement” was mostly unmatchable	1,491 of 1,642 could not match	observatory	measured
No accuracy difference detected between Jev and the best judges (after correction); Jev cheapest	55/63 correct	judge benchmark	preregistered
No judge useful on real read decisions	0 of 10 judges	trace study	preregistered
Routing on Fable saves	$0.56 \times (0.49-0.64)$	main-v1	preregistered
Routing on Fable saves, fresh scenarios	$0.581 \times (0.542-0.625)$	S1, H1	preregistered, replicated
Routing on Opus costs more	$1.255 \times (1.197-1.321)$	S1, H5	preregistered, replicated
Rule R* equals the decision model at session start	$1.008 \times (90 \% 1.000-1.019)$	S1, H2	preregistered
Bundle cost-equivalent on Opus within the preregistered ± 5 % margin	$0.999 \times (90 \% 0.984-1.017)$	S1, H3	preregistered
Medium effort: Sonnet	$0.821 \times (0.781-0.861)$	follow-up 1	preregistered, provenance-limited
Medium effort: Fable	$0.860 \times (0.833-0.885)$	follow-up 2	preregistered
Medium effort on Opus: saving not demonstrated	$0.992 \times (0.974-1.009)$	S1, H4	preregistered
Noninferiority not established for review/explain on Sonnet	lower bound -0.063	S1, H7	preregistered
Live decide-once equals its prediction	$1.014 \times (90 \% 0.996-1.032)$	S1, H6	preregistered
Effort switches rewrite the cache	$12.9 \times$ cache writes	main-v1 re-analysis	exploratory
Keyword proxy keeps quality at a cost	$0.749 \times$, turn-pass 0.002	S1 replay	exploratory

2 Telemetry shows that decisions are fast, not whether they are right

The question at this stage: before building anything new, what could the bundle’s own records tell us about whether its decisions help? The fast-decisions bundle records every decision it makes, privately and locally, so we read what was already there: 464,524 events in 4,470 session files, recorded from 2026-09-17 to the 2026-10-05 snapshot cutoff; 171,609 of them fall in the first week (2026-09-17 to 2026-09-23). 398 sessions were real work; 1,136 were tests.

Decisions are fast.. Figure 2 puts the decision calls next to the thing they try to save. A local Qwen3 model proposed a file read in 48 ms at the median; Jev judged a session’s starting model in 159 ms (p95 311 ms). A host model call took 4.8 s at the median and 26.9 s at p95. Speed is not the obstacle.

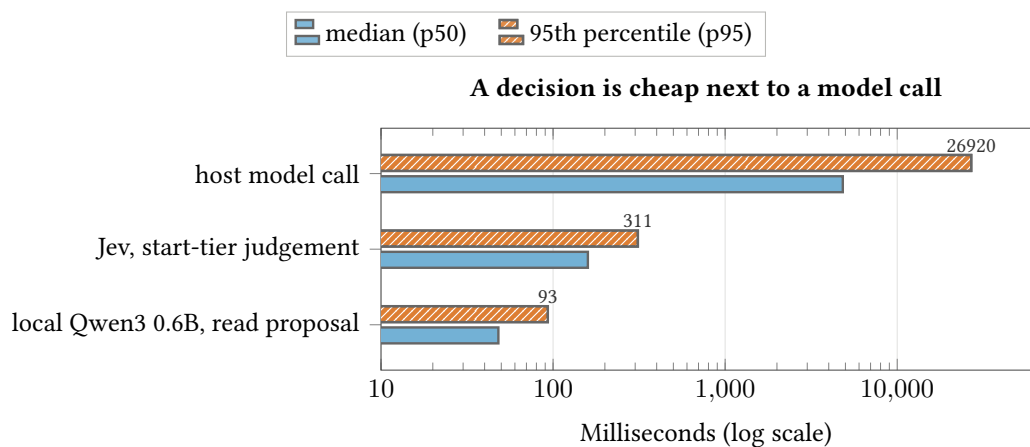


Figure 2: A decision call is two orders of magnitude faster than a host model call. Latency of decision calls and of host model calls, measured by the bundle’s recorder (log scale).

How to read it. Each row has a median bar and a 95th-percentile bar. The decision rows are two orders of magnitude shorter than a host model call; the p95 bars show the slow tail.

The 7.1 % that was not what it seemed.. The telemetry reported that the read-shortcut proposals agreed with the host’s next action only 7.1 % of the time (117 of 1,642). Taken at face value, that condemns the idea. Decomposed, it says little about the decision model:

- 1,491 of the 1,642 decisions followed a host call to a tool the proposal could not contain, so they could never match. “Agreement” was partly a labelling artefact.
- 648 were scored by a synthetic demo backend that always picks the first candidate.
- Where a match was possible, the real local scorer agreed with the host 77.5 % of the time (117 of 151; 95 % CI 70.2–83.4 %).

Agreement is still not correctness: the store keeps no state text, so whether a disagreement was a mistake cannot be judged from it.

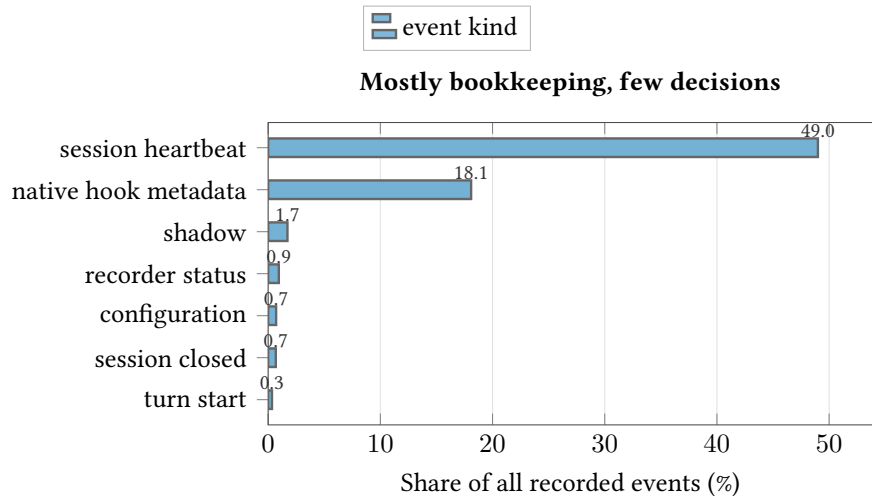


Figure 3: Most recorded events are bookkeeping; decision-bearing events are a thin slice. What the observatory store holds: the seven most common kinds of recorded event, as a share of all events.

How to read it. The longest bar, session heartbeat, is 49.0 % of everything recorded. Decision-bearing events are a thin slice. A store this dominated by bookkeeping can tell you that decisions happen, not whether they were right.

Claimed savings were estimates.. The bundle’s own receipts claimed \$222.66 saved by keeping caches warm (194 production receipts) and showed per-step routing to Sonnet costing money (\$−1.80). The read shortcut “would have” avoided 117 host turns in 994 decisions. None of these was measured as a saving; later measurement turned the per-step routing off. In production, the first start-tier decision of a session was most often the scope gate keeping the host (105 of 214 sessions, 49 %); Jev judged 68 sessions strong and 35 cheap.

Key idea. Telemetry told us decisions are cheap and fast, and that the bundle’s own savings claims were estimates. It could not tell us whether decisions were right or whether they saved money. That needed labelled cases and paired sessions.

What this left open. Whether a decision is right needs a label that telemetry does not keep, so the next stage built labelled cases (Section 3).

3 On labelled decisions Jev is the best value; on real reads no judge is safe

The question at this stage: telemetry could not say whether a decision was right, so: on cases where the right answer is known, which decision model is right most often, at what price and speed? To know whether a decision is right we need labelled cases. We built two benchmarks and graded every decision model under the gate the bundle ships: act only when the top probability is at least 0.90 and leads the runner-up by at least 0.20, within 3 s; otherwise fall back to the host.

Constructed cases.. 10 judges answered 90 development cases (a screen) and 63 fresh holdout cases whose analysis was preregistered, 3 times each: file reads, searches, selections and computer-use questions, including cases with side effects or prompt injections. Two blind reviewers checked the labels (Cohen’s $\kappa = 1.00$ with the final labels). On the holdout, Jev was right on 55 of 63 cases at \$18.41 per million decisions and a p95 of 222 ms. GPT-6 Luna (60) and GPT-6.1 Sol (62) were

slightly more accurate, but neither difference survived the preregistered Holm correction (adjusted $p = 0.16$ and 0.09), and both were slower and dearer ($p95$ 1,850 and 4,252 ms). Cloudflare’s Clef, tested later and labelled post hoc, matched Jev’s accuracy (57 against an in-run Jev 56) at $3.2\text{--}4.4\times$ the cost per decision and a $p95$ of 663–738 ms from our machine.

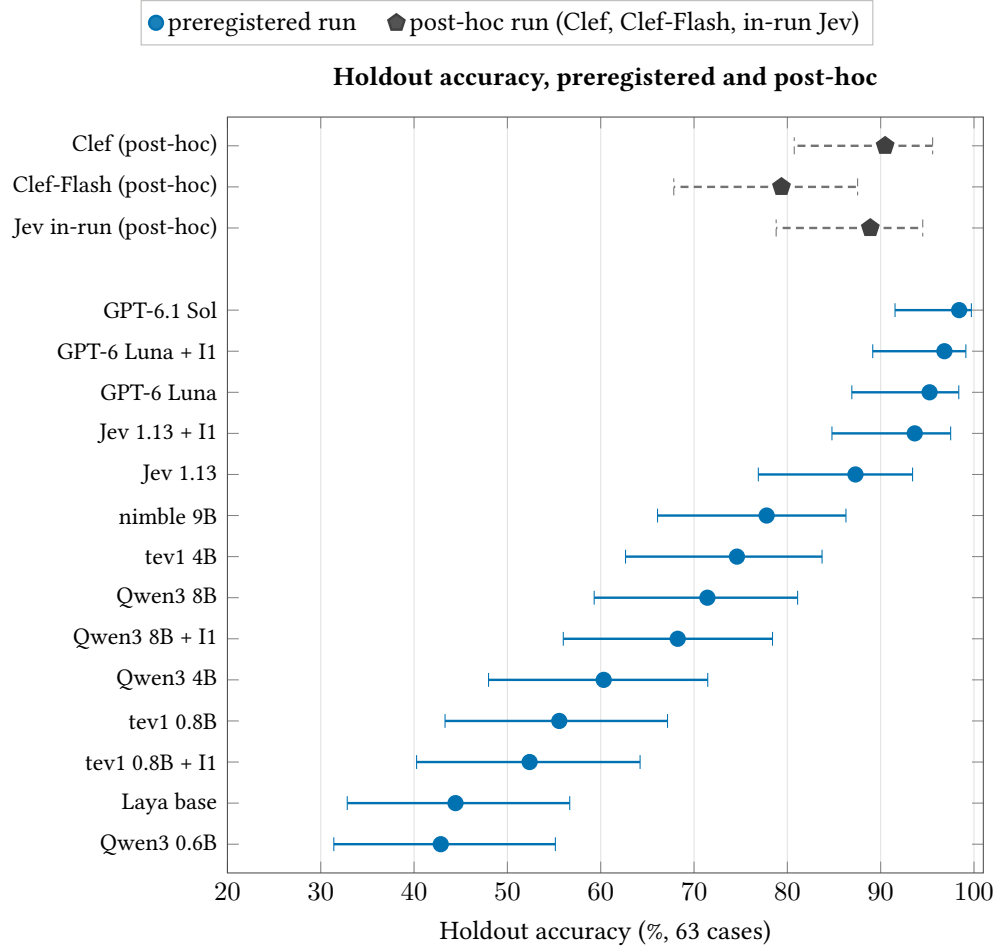


Figure 4: No accuracy difference between the top judges was detected after the preregistered correction. Holdout accuracy with 95 % Wilson intervals: the preregistered run, then (below the gap) the post-hoc run of Clef, Clef-Flash and the in-run Jev reference.

How to read it. Each row is a judge; the dot is its accuracy and the whisker its 95 % interval. The top judges’ whiskers overlap: no accuracy difference was detected after the preregistered correction (which is not the same as showing the judges equivalent). Jev sits among them at a fraction of the cost and latency of the cloud models (companion report, Part I).

Real decisions.. Constructed cases can be easier or harder than real ones. A second preregistered study rebuilt 42 real read-shortcut decisions from earlier sessions, exactly as the in-session judge had seen them. Reviewers agreed with each other ($\kappa = 0.84$) much more than with what the host actually did next ($\kappa = 0.51 - 0.57$). The preregistered usefulness rule asked for at least 4 correct automatic reads with few misfires. 0 of 10 judges met it (Fig. 6). Jev took the most correct reads (10 of 13) but also made 5 wrong automatic reads.

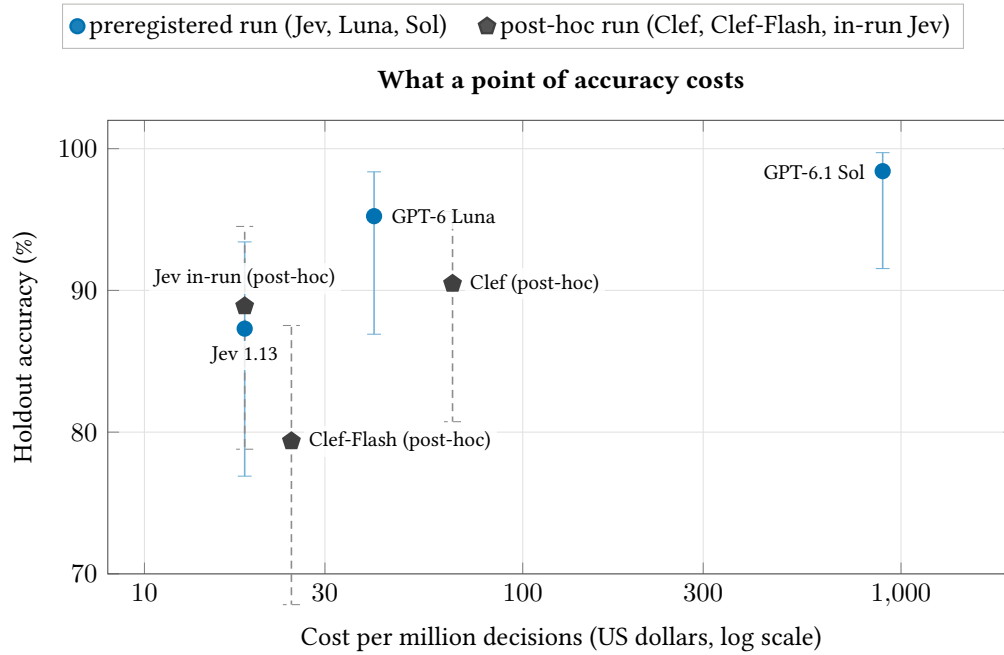


Figure 5: Jev is the cheapest judge near the top of the measured accuracy range. Cost per million decisions (log scale) against holdout accuracy, for the judges that have a price. Blue: preregistered run; black pentagons: post-hoc run (Clef, Clef-Flash and the in-run Jev reference).

How to read it. Left is cheaper, up is more accurate. Jev is the cheapest point near the top; the cloud models buy a few points of measured accuracy, a difference not detected after correction, for many times the price.

3.1 OpenAI Decisions API (post hoc)

OpenAI’s Decisions API is a native endpoint for exactly this kind of typed question: GPT-6 Luna returns a probability per allowed answer. We ran it after the preregistered work, so everything here is post hoc and exploratory, against an in-run Jev 1.13 reference on the same cases: three repetitions in two option orders, the bundle’s gate with its 3 s deadline, and the per-case majority over repetitions (Table 2 and Fig. 7).

- **Accuracy.** Luna was right on 88/90 development cases (Jev 86/90), 59/63 holdout cases (Jev 55/63), 16/21 real trace-dev decisions (Jev 16/21) and 30/42 trace-holdout decisions (Jev 23/42). Only the trace-holdout accuracy difference survived the Holm correction ($p = 0.039$; exploratory).
- **Misfires.** Luna’s wrong-automatic rate was no higher than Jev’s on every split; none of these differences was significant. Under the trace usefulness rule, neither judge was useful on either trace split.
- **Speed.** OpenAI’s servers reported 59–62 ms at the median and 136–179 ms at p95, but from our machine the client saw 1.4–1.8 s at the median and 3.0–4.8 s at p95. The network was slow in this run for both providers: Jev’s client median was 1.4–1.6 s, against 0.15–0.17 s on 2026-10-04. The comparison between the two judges is paired in time; the absolute latencies are not comparable with earlier evidence.

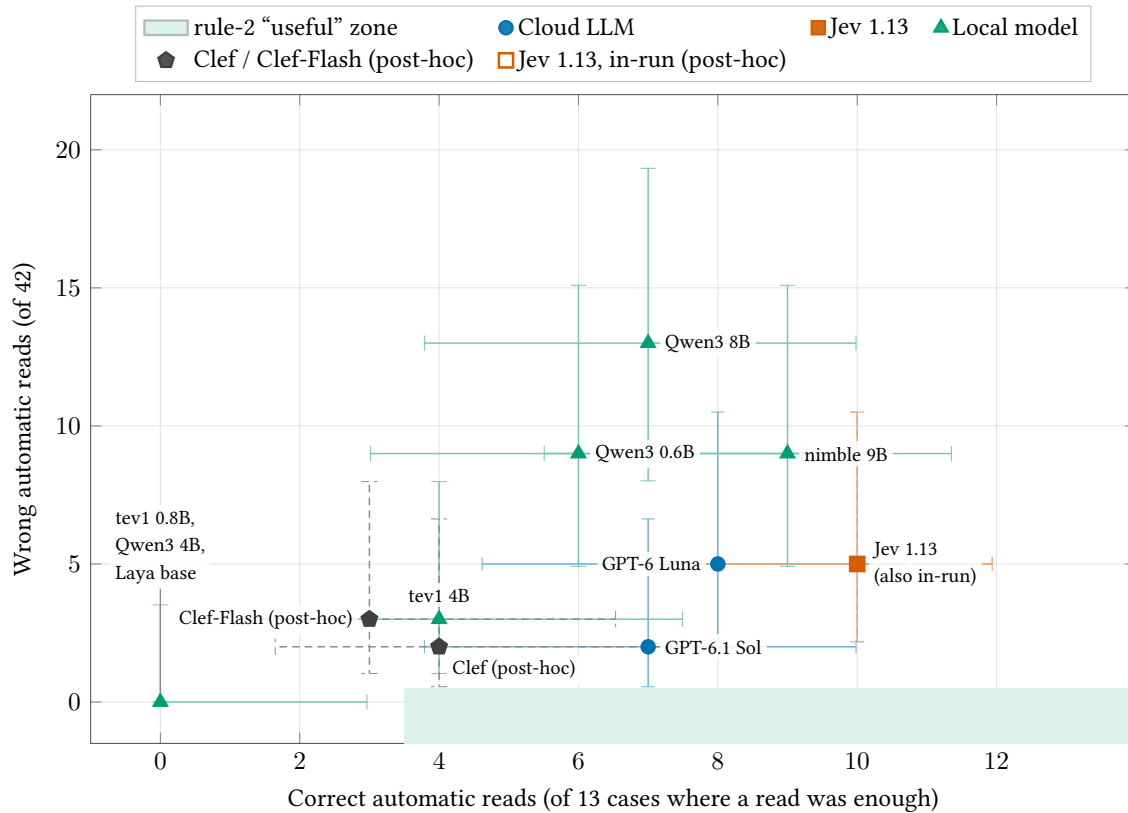


Figure 6: On real read decisions, no judge reached the usefulness zone. Real decisions: correct automatic reads against wrong automatic reads on trace-holdout, with 95 % Wilson intervals: the preregistered run, plus the post-hoc Clef arms (black pentagons). The shaded strip is where the preregistered rule would call a judge useful.

How to read it. Right is more saved host calls; up is more misfires. No judge reaches the shaded strip: every judge with enough correct reads also misfired.

- **Cost and reliability.** \$20–76 per million decisions against Jev’s \$16–46. 9 of 1,296 Luna requests failed in transport.

Key idea. A native typed-decision endpoint from OpenAI matched or exceeded Jev’s accuracy at a similar price on our cases. Its servers answer in tens of milliseconds, but from here each call took seconds. It does not change the session-start finding: a one-line rule matched the decision model there.

Key idea. Jev is the best value among the decision models we tested: no accuracy difference from the expensive ones was detected after the preregistered correction, and it is much cheaper and faster. But on real read decisions no model was safe enough to act on its own, so the read shortcut stays off. The money, if any, had to come from a bigger decision: which model runs the session.

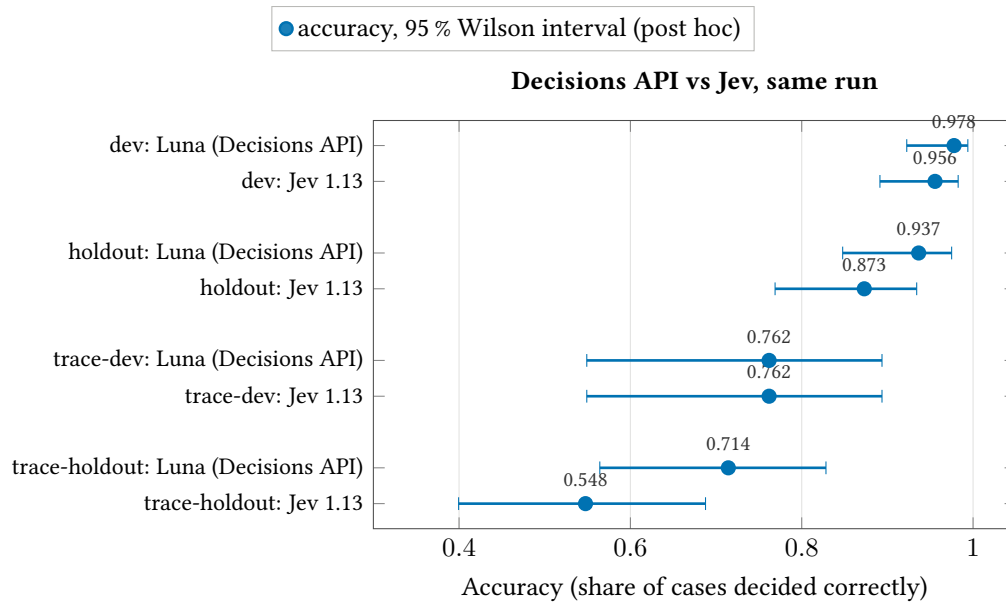


Figure 7: The Decisions API was at or above Jev on every split (post hoc). OpenAI Decisions API (GPT-6 Luna) against the in-run Jev reference: per-case majority accuracy with 95 % Wilson intervals on the four splits (post hoc).

How to read it. Each pair of rows is one split. Luna is at or above Jev on every split; only the trace-holdout gap survived the correction. Overlapping whiskers mean no difference was detected, not that the judges are equivalent.

Table 2: OpenAI Decisions API (post hoc) against the in-run Jev reference: correct decisions, wrong automatic actions, coverage, client latency (from our machine, slow network in this run) and dollars per million decisions.

Split	Judge	correct	wrong auto. (%)	coverage (%)	p50 s	p95 s	\$ / 1M
dev	GPT-6 Luna, Decisions API	88/90	0.0	71.1	1.80	4.80	20.3
dev	Jev 1.13 (in-run reference)	86/90	3.3	75.6	1.46	4.46	15.9
holdout	GPT-6 Luna, Decisions API	59/63	1.6	68.2	1.47	3.44	25.6
holdout	Jev 1.13 (in-run reference)	55/63	1.6	63.5	1.57	3.73	18.4
trace-dev	GPT-6 Luna, Decisions API	16/21	9.5	47.6	1.68	3.24	71.8
trace-dev	Jev 1.13 (in-run reference)	16/21	14.3	66.7	1.40	4.07	43.2
trace-holdout	GPT-6 Luna, Decisions API	30/42	2.4	16.7	1.41	3.00	76.2
trace-holdout	Jev 1.13 (in-run reference)	23/42	11.9	35.7	1.39	3.44	46.1

What this left open. A judge can be accurate and still save nothing. The only decision left that clearly moves money is which model runs the whole session, and that can only be measured in whole sessions ([Section 4](#)).

4 In whole sessions, routing pays on Fable and costs on Opus

The question at this stage: the judge benchmarks left one decision that clearly moves money, the model a whole session runs on. Does routing that choice to a cheaper model actually save money once caching and effort are counted? The decision that moves money is the session’s starting model. To measure it we ran whole multi-turn sessions, in pairs: the same scripted task, started together from the same frozen workspace, once on the plain host and once under a routing policy, each with its own cache key. The main campaign (main-v1) ran 1,036 sessions on 70 scenarios of 8 to 16 turns, with Fable 5.1 and Opus 5.5 as hosts. Hypotheses, thresholds and the split were preregistered; the estimator was written after the data, which the companion report states plainly.

Why caching made this necessary.. One of us (DK) asked whether routing still saves money in longer chats. Prompt caching makes re-sending a conversation cheap on the model that cached it, and switching models throws that cache away. An earlier survey had found the shipped router’s Opus-host ratio going from $0.95\times$ in single-turn sessions to $1.31\times$ in 4-turn ones. Estimates from receipts could not settle it.

A *paired cost ratio* compares two sessions of the same task. For the running example, click-flow-explain in S1, the routed sessions cost \$2.34 against \$4.04 on plain Fable: a ratio of $0.580\times$. The results below average such ratios over many tasks ([Section 6.2](#) works through the arithmetic).

What it found.. With Fable as host, choosing Sonnet once at the start of the session (*sticky*) cost $0.56\times$ the plain host (95 % CI 0.49–0.64; a typical per-session saving of 44 % as a geometric mean, and an actual spend cut of 37.3 % over all scenarios). Quality stayed inside the preregistered margin on the test split only; over all scenarios the margin was not shown (-0.021 , CI -0.058 to 0.012), and 3 task types lost more (review -0.155 , explain -0.143 , feature -0.071). With Opus as host, every routing arm cost more (1.25 – $1.43\times$): Opus’s cache reads are cheap enough that leaving them costs more than Sonnet saves. That conclusion depends on the price table; Opus routing would break even if its cache reads cost about \$0.38 per million tokens instead of \$0.20.

The effort confound.. An external peer reviewer noticed that routed Sonnet ran at medium reasoning effort while the plain-Sonnet control ran at the provider default. Two follow-ups measured effort directly. Plain Sonnet at medium cost $0.821\times$ its default (0.781 – 0.861 ; designed in advance, provenance-limited), and plain Fable at medium cost $0.860\times$ its default (0.833 – 0.885 ; preregistered), both with no detectable turn-pass loss on their 23 scenarios. Medium effort independently reduced Fable cost by about 14 % in the follow-up; routing remained the larger cost lever: sticky-on-Sonnet cost about $0.65\times$ what Fable at medium did (exploratory, not concurrent). The two review rounds and their responses are recorded with the companion report.

Key idea. Measured, not estimated: on Fable, deciding once to run the session on Sonnet saved about half the cost, with a quality caveat on some task types. On Opus it cost more. Lowering reasoning effort is a second, smaller lever that works on both Sonnet and Fable.

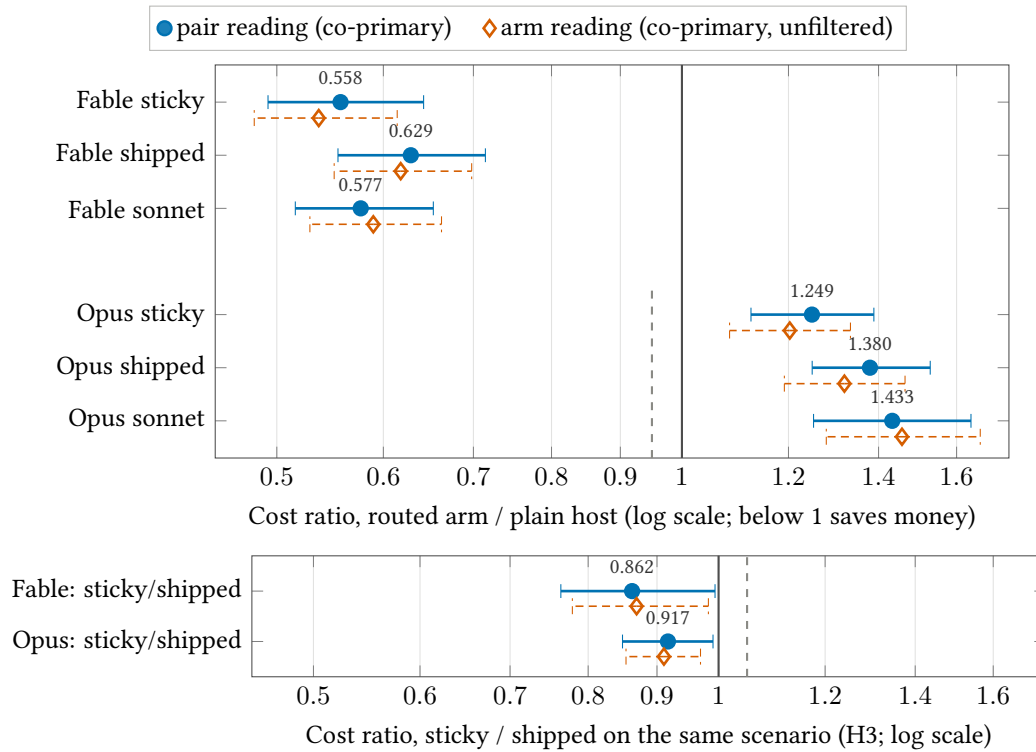


Figure 8: On the main-v1 test split, every Fable routing arm saved money and every Opus arm cost more. Confirmatory cost ratios on the main-v1 test split. Top: each routed arm against the plain host. Bottom: the sticky arm against the shipped arm on the same scenarios. Whiskers are 95 % intervals; the dashed lines mark preregistered thresholds.

How to read it. The dot is the geometric-mean cost ratio, the whisker its 95 % interval, on a log axis. Rows left of 1 saved money. All Fable rows are well left of 1; all Opus rows are right of it.

What this left open. The saving came with a decision model choosing the session's model. Was the model needed for that choice, or would a simple rule do as well? And would the result hold on tasks no one had tuned on? [Section 5](#) answers both.

5 At session start, a one-line rule matches the decision model

The question at this stage: whole sessions showed that routing pays on Fable, with a decision model making the choice. Does that choice need a model at all, and which configuration should ship? By now the open question was no longer which decision model is best, but whether the session-start decision needs a model at all, and how to configure it.

A counterfactual on the existing sessions.. A decide-once policy makes one choice per session, so every policy’s cost can be read off the paired sessions already run: the plain host where it keeps the host, the routed session where it routes. [Figure 9](#) and [Table 3](#) price every such policy on all main-v1 scenarios. On Fable, Jev routed 89 % of sessions and cost $0.545\times$ the plain host. The simple rule R^* (route unless the workspace has more than 300 files) routed 99 % and cost $0.509\times$ at the same turn-pass difference (-0.019). Jev’s “keep on host” calls bought nothing: in every scenario-rep where Jev kept the host, the routed session scored the same. Jev and R^* disagreed on only 14 of 140 scenario-reps, which bounds how much they can differ. On Opus the price gate, which never routes there, sat at the oracle (regret \$1 per 1,000 sessions), while ungated Jev would have cost $1.159\times$.

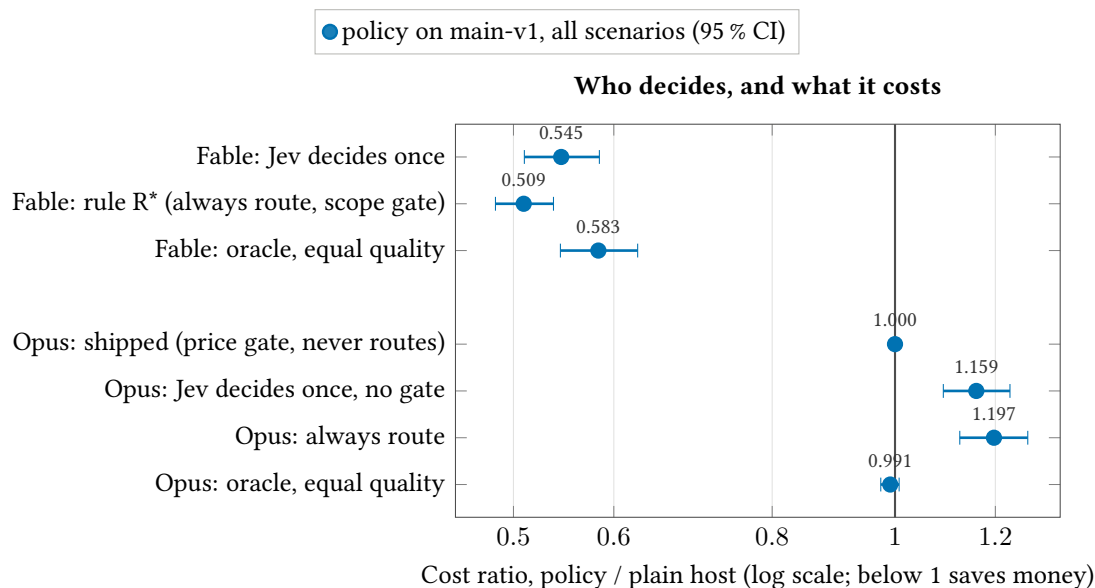


Figure 9: On Fable the rule R^* is at least as cheap as Jev; on Opus every routing policy costs more. Decide-once policies priced on the existing main-v1 sessions (all scenarios, both reps), with 95 % scenario-cluster intervals. The oracle chooses with hindsight on the other rep, at equal quality. Exploratory.

How to read it. Each row is a policy. Left of 1 saves money. On Fable, the rule R^* is at least as cheap as Jev; on Opus, every policy that routes costs more than keeping the host.

An artefact to avoid.. Sessions that changed reasoning effort request by request rewrote the prompt cache each time: within a turn, a request after an effort change wrote 11,650 tokens to the cache on average against 900 when effort stayed the same ($12.9\times$). Effort should be set once per session, like the model.

The core study (S1) [preregistered].. These findings were frozen into candidate configurations and a preregistered holdout study on 60 scenarios disjoint from main-v1, smoke-tested before preregistration: does the frozen configuration hold up, does the decision model beat the rule, and does medium effort help Opus? A deterministic freeze rule then picks each host’s configuration.

Table 3: Decide-once policies on main-v1 (exploratory): share routed, geometric-mean cost ratio and turn-pass difference against the plain host, and dollars per 1,000 sessions.

Policy (decided once per session)	routed	cost ratio [95 % CI]	turn-pass Δ [95 % CI]	\$ / 1,000
Fable: always host (plain)	0 %	1.000 [1.000, 1.000]	0.000 [0.000, 0.000]	5,311
Fable: Jev decides once	89 %	0.545 [0.510, 0.585]	−0.019 [−0.052, 0.010]	3,108
Fable: rule R* (always route, scope gate)	99 %	0.509 [0.484, 0.538]	−0.019 [−0.052, 0.010]	2,739
Fable: oracle, equal quality	81 %	0.583 [0.544, 0.627]	0.012 [−0.004, 0.030]	3,268
Opus: shipped (price gate, never routes)	0 %	1.000 [1.000, 1.000]	0.000 [0.000, 0.000]	2,125
Opus: Jev decides once, no gate	89 %	1.159 [1.092, 1.232]	−0.009 [−0.033, 0.013]	2,498
Opus: always route	100 %	1.197 [1.125, 1.273]	−0.005 [−0.030, 0.019]	2,620
Opus: oracle, equal quality	22 %	0.991 [0.975, 1.008]	0.000 [−0.011, 0.009]	2,124

How the panel was prepared.. Before the preregistration was committed, one plain-Sonnet smoke session ran on each of the 60 scenarios (\$182.80). It found prompt or grader defects in 30 scenarios: prompts that did not state what a check required, and checks that rejected correct answers. These were repaired; genuine agent mistakes were left as they were; every scenario was then revalidated mechanically (the starting workspace fails, the reference passes, wrong variants fail). The smoke results enter no hypothesis and are not pooled with S1.

S1 methods in brief. Quality. Each scripted turn carries automated checks (tests, file patterns, required facts or document sections); the *turn-pass fraction* is the share of a session’s turns whose checks passed, and a final-state check on the hidden tests is reported as additional evidence. The graders were validated before the run: the starting workspace must fail, the reference solution must pass every turn, and plausible wrong answers must fail. Non-inferiority means the 95 % lower bound of the turn-pass difference is above −0.05, that is, no worse than the specified 5-point margin. These are claims about these automated checks, not about human judgement of, for example, the quality of a code review or an explanation. **Cost.** The primary cost is the tools-normalized list-price cost of each session, plus estimated decider charges where a decider runs. The normalization reprices the first shared tool-definition prefix from the cache-write to the cache-read rate; it changed the accepted sessions’ recomputed cost by 0.45 % in main-v1 and 0.135 % in S1 (data dictionary in the evidence package).

Composed policies. S1 executed pinned-host, pinned-Sonnet and shipped-policy sessions. H1 and H2 evaluate decide-once policies by selecting the corresponding pinned outcome for each scenario and repetition: the rule or recorded Jev decision determines which session contributes its cost and quality. These policies therefore reuse executed outcomes rather than running a separate session for every configuration. Intervals resample whole scenarios. H6 separately compares live shipped sessions with their matched reconstructions, providing an empirical check of this cost decomposition (not of every quality property of every reconstructed policy).

S1 ran 1,224 sessions on 60 scenarios. [Table 4](#) lists the verdicts; in plain words:

- **H1 (Fable value), supported.** The frozen Fable configuration cost $0.581 \times$ plain Fable (95 % CI 0.542–0.625) with turn-pass −0.013 (−0.028 to 0.002). The saving is real, but smaller than predicted: the frozen prediction from main-v1 was 0.508 (0.483–0.536), and the observed ratio lies above that interval.

Table 4: S1 (holdout-v3) preregistered hypotheses. Cost ratios are geometric means over scenarios; Holm within the two preregistered families. H3b is descriptive.

Hypothesis	cost ratio (95 % CI)	turn-pass Δ (95 % CI)	Holm p	verdict
H1	0.581 (0.542–0.625)	−0.013 (−0.028–0.002)	< 0.001	✓ supported
H2	1.008 (1.000–1.022)	−0.003 (−0.008–0.000)	< 0.001	✓ supported
H3	0.999 (0.982–1.021)	0.007 (−0.004–0.022)	< 0.001	✓ supported
H3b	1.004 (0.987–1.022)	0.004 (−0.002–0.010)	–	descriptive
H4	0.992 (0.974–1.009)	−0.002 (−0.013–0.009)	0.187	✗ not supported
H5	1.255 (1.197–1.321)	−0.004 (−0.024–0.017)	< 0.001	✓ supported
H6	1.014 (0.993–1.035)	0.004 (−0.011–0.019)	< 0.001	✓ supported
H7	0.616 (0.565–0.672)	−0.028 (−0.063–0.013)	0.120	✗ not supported

- **H2 (decision model against a rule), supported.** Letting Jev choose the session’s model and letting the one-line rule R^* choose it were equivalent: $1.008 \times$ (90 % CI 1.000–1.019), turn-pass −0.003. They disagreed on only 4 of 120 scenario-reps. The preregistered decision follows: ship the simpler rule R^* , with no external call and no consent needed; Jev becomes an opt-in decider.
- **H3 (bundle overhead), supported.** The shipped bundle on Opus cost $0.999 \times$ plain Opus (90 % CI 0.984–1.017): cost-equivalent within the preregistered $\pm 5\%$ margin. On Fable, the bundle with the host pinned cost $1.004 \times$ (descriptive).
- **H4 (Opus at medium effort), not supported.** Opus at medium cost $0.992 \times$ its default (95 % CI 0.974–1.009): medium effort did not demonstrate a saving, unlike Sonnet ($0.821 \times$) and Fable ($0.860 \times$).
- **H5 (routing on Opus), supported.** Routing to Sonnet still lost on Opus: $1.255 \times$ (1.197–1.321), on fresh scenarios and without the effort-switch artefact.
- **H6 (live equals predicted), supported.** Live decide-once sessions cost what the decomposition predicted ($1.014 \times$, 90 % CI 0.996–1.032).
- **H7 (review and explain on Sonnet), not supported.** On the 20 review and explain scenarios (19 for cost, after the gate-failed scenario is excluded), the turn-pass lower bound for Sonnet against the host was −0.063, below the −0.05 margin: non-inferiority was not established (the interval also includes zero, so a loss is not shown either). The preregistered rule therefore keeps these task types on the host (keep_on_host: [review, explain]).

What the freeze rule chose.. On Fable the rule picked `always_route | strong default | scope off | keep none (0.528×)`, not the preregistered `rstar | strong medium | scope 300 | keep none`. That makes it *selected on holdout*: it must replicate in S2 before it ships. It also conflicts with H7, which keeps review and explain on the host. What ships for Fable until S2 replicates is therefore the preregistered configuration plus `keep_on_host: [review, explain]`, applied through a keyword proxy (Section 7). On Opus no candidate qualified, so Opus keeps the shipped defaults: no routing, default effort.

Key idea. The decision model's start-of-session choice was matched by a one-line rule on these hosts and prices. Where the money is, routing on Fable and never on Opus, price arithmetic and a rule decide it. The value of a system-1 model is not in this decision.

Process note: how the S1 run went. The study spent \$4,975.51. 8 Fable sessions of one scenario (bleach-sanitize-review) were served partly by Opus, most likely the provider's fallback (an inference); the preregistered rule excluded them from cost, so Fable cost contrasts use 59 scenarios. The stop rule for gate failures was not followed to the letter, because the gate is evaluated when rows are extracted, not live; this is a disclosed deviation. A Forge outage made 135 of 376 wave attempts (36 %) fail on infrastructure; every failed wave was rerun whole. The budget cap was raised from \$5,000 to \$6,300 during the run, though the final spend stayed under the original cap, and concurrency changed several times. The A/A copies differed in turn-pass by -0.047 on Opus (11 pairs), so small quality differences cannot be interpreted. Rerunning the analysis without the flagged scenarios changed no verdict and no freeze choice.

What this taught us, and what it left open. At the session start the decision model added nothing that a one-line rule and price arithmetic did not already provide: routing pays on Fable and costs on Opus, and the rule and the model choose the same way. Section 6 opens the S1 results up to show where that answer is solid and where it is thin.

6 Inside S1: where the result is solid and where it is thin

This section opens up the S1 results one picture at a time. All of it is the preregistered S1 data; anything not preregistered is labelled exploratory.

6.1 Every hypothesis on one page

Figure 10 puts all eight preregistered comparisons side by side, cost on the left and quality on the right, so the reader can see at once which results move money and which are guards.

Key idea. Two of the eight rows carry money: H1 (Fable routing saves about two fifths) and H5 (Opus routing loses about a quarter). The rest are guards: the bundle is cost-equivalent within the $\pm 5\%$ margin (H3), the model adds nothing over the rule (H2), live sessions behave as predicted (H6), and review and explain need protection (H7).

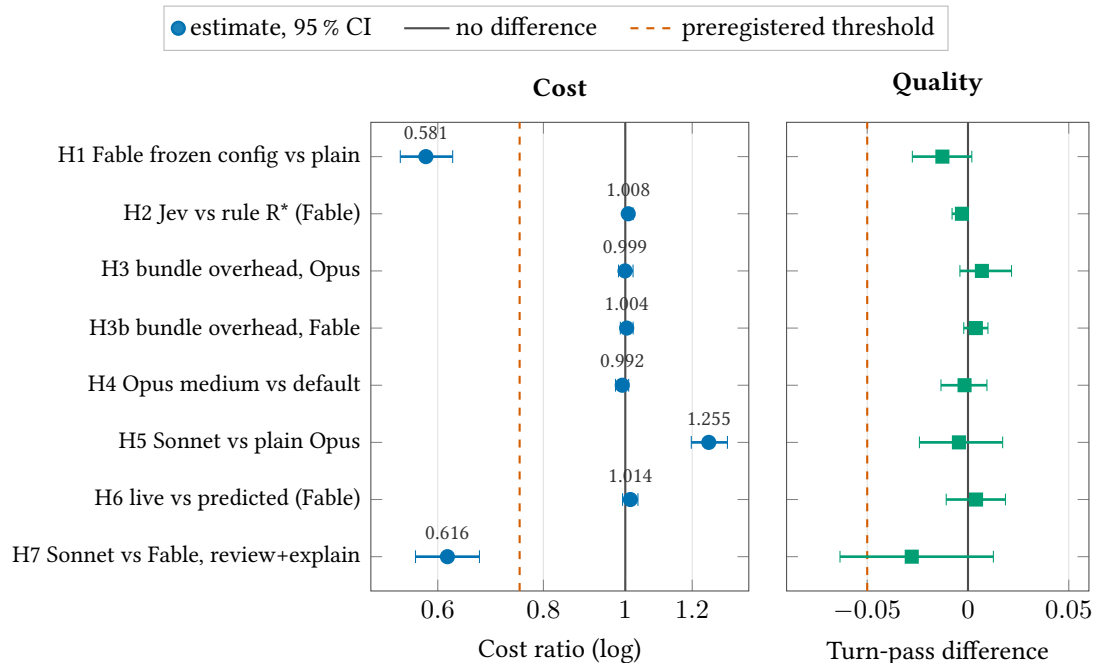


Figure 10: Two hypotheses carry money (H1, H5); the rest are guards. S1 hypotheses H1–H7: cost ratio (left, log scale) and turn-pass difference (right), each with its 95 % interval. H3b is descriptive. Thresholds: the H1 cost bound (0.75) and the quality margin (−0.05).

How to read it. Read each row across both panels. A row helps a configuration when its cost dot is left of 1 and its quality whisker stays right of the dashed line. H1 does both. H5 is far right of 1: routing on Opus costs more. H4 straddles 1: medium effort did nothing measurable on Opus. H7’s quality whisker crosses the dashed line, which is why review and explain stay on the host.

6.2 How to read a paired cost ratio

Worked example: one scenario, two reps, one ratio. Take click-flow-explain, the S1 scenario whose ratio sits closest to the overall result. Under the frozen Fable configuration it was routed to Sonnet. Across its 2 repetitions the routed sessions cost \$2.34 on average and the plain Fable sessions \$4.04. Its paired cost ratio is therefore $2.34/4.04 = 0.580$; on the log scale that is $\ln 0.580 = -0.544$. Doing the same for each of the 59 scenarios, averaging the logs and exponentiating gives the *geometric mean*, 0.581: the H1 headline. The interval comes from resampling whole scenarios 10,000 times and repeating the calculation.

Why a geometric mean? A ratio of 0.5 and a ratio of 2 should cancel; their arithmetic mean (1.25) says “more expensive”, their geometric mean (1) says “no change”. Averaging logs treats halving and doubling symmetrically.

6.3 The variance underneath the headline

A geometric mean hides how much individual tasks vary. [Figure 11](#) shows every scenario’s paired ratio, sorted, for the two results that carry money.

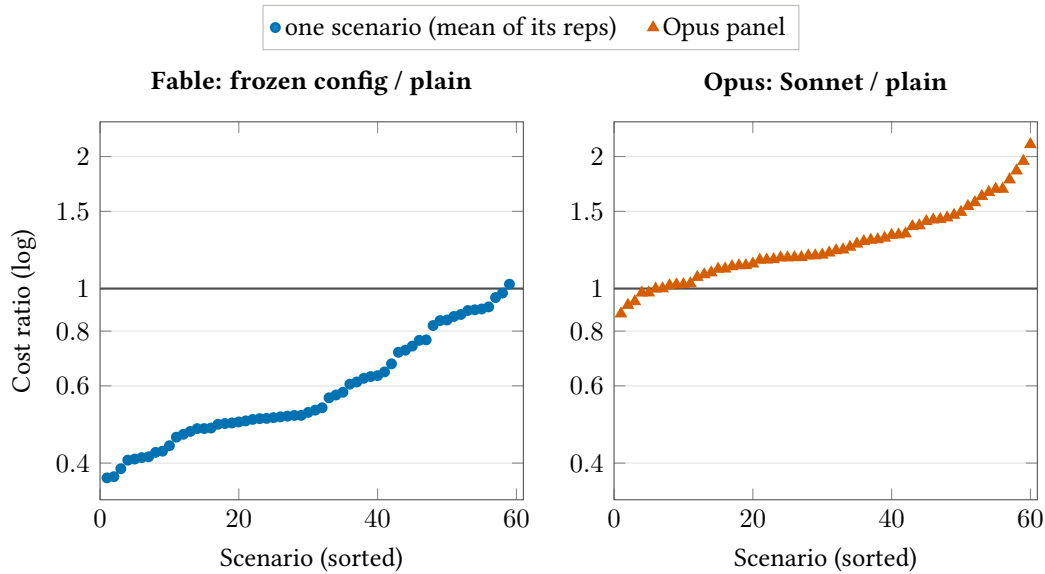


Figure 11: The direction is consistent across scenarios; the size varies widely. Per-scenario paired cost ratios in S1, sorted. Left: the frozen Fable configuration against plain Fable (59 scenarios). Right: Sonnet decided once against plain Opus (60 scenarios). Log scale.

How to read it. Each dot is one scenario. Below the line saves money. On Fable, 1 of 59 scenarios cost more than plain Fable, and the ratios run from 0.37 to 1.02. On Opus, 53 of 60 cost more, from 0.88 to 2.13. The direction is consistent; the size varies a lot from task to task.

6.4 Where the risk sits: task types and families

The quality cost of routing is not spread evenly. [Figure 12](#) splits the Fable result by task type and by scenario family (exploratory).

6.5 Which configuration the rule picked

The freeze rule considered every configuration the preregistration listed and chose one per host. [Figure 13](#) shows them all, with the choice and the preregistered configuration marked; [Table 7](#) in the appendix lists them.

6.6 The decision model against the rule

Jev and the rule R^* chose differently on only 4 of 120 Fable scenario-reps ([Table 5](#)).

Worked example: why the two deciders cannot differ much. Two deciders that agree on a scenario-rep produce the same session there, so their difference comes only from the scenario-reps where they disagree. The share of those is $d = 4/120 = 0.033$. On them the choices differed in cost by \$1.01 per session on average. So the most the deciders can differ, averaged over all sessions, is $d \times \$1.01 = \0.034 per session, about 1.2 % of a typical session (\$2.87). That bound, not a significance test, is what makes “equivalent” believable: with so few disagreements, no amount of skill on those few can move the total.

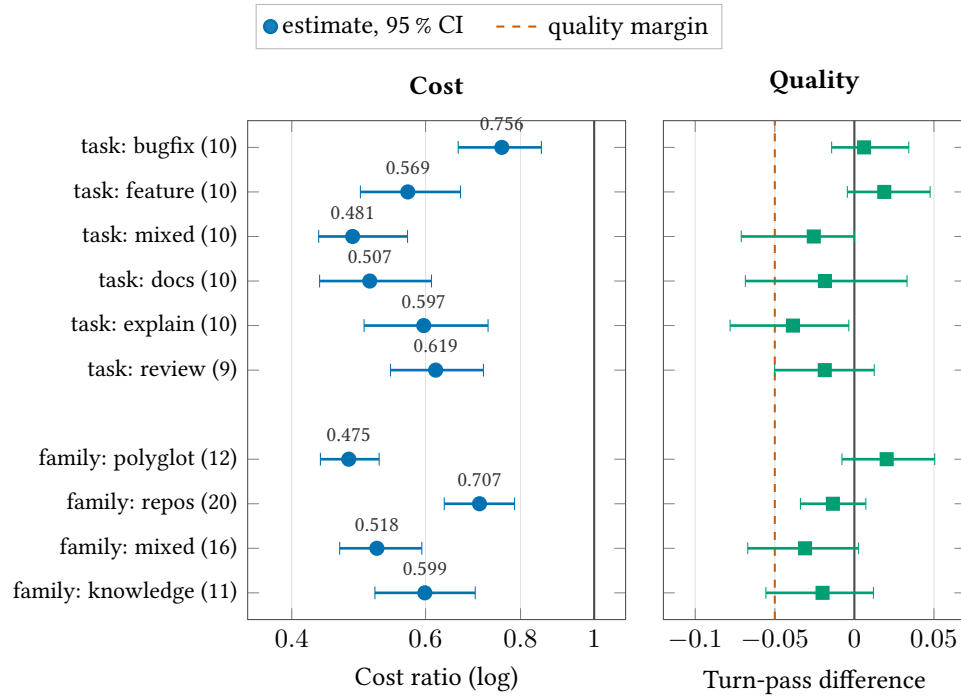


Figure 12: Every subgroup saves money; the quality risk concentrates in explain and mixed work, with review at the margin. The frozen Fable configuration against plain Fable, split by task type and by scenario family (number of cost-valid scenarios in brackets; the quality panel is unfiltered and uses every scenario of the subgroup), with 95 % intervals. Exploratory: subgroups were not preregistered except H7.

How to read it. Every subgroup saves money. The quality panel is where they differ: explain (−0.039; lower bound −0.078) and mixed work cross the margin, review touches it (−0.050), while bugfix and feature sit near zero. Small subgroups give wide whiskers; treat single rows with caution.

Table 5: The 4 S1 scenario-reps on which the decision model (Jev) and the rule R* chose different session models, with what each choice cost and how it scored.

Scenario	rep	model chosen by		session cost		turn-pass	
		Jev	rule R*	Jev	R*	Jev	R*
go-bowling	1	Fable, medium	Sonnet (Pc)	\$6.53	\$4.06	0.857	1.000
go-bowling	2	Fable, medium	Sonnet (Pc)	\$5.66	\$4.77	0.929	0.929
peewee-filterfk	1	Fable, medium	Sonnet (Pc)	\$2.15	\$1.94	0.889	1.000
peewee-filterfk	2	Fable, medium	Sonnet (Pc)	\$2.34	\$1.87	0.889	1.000

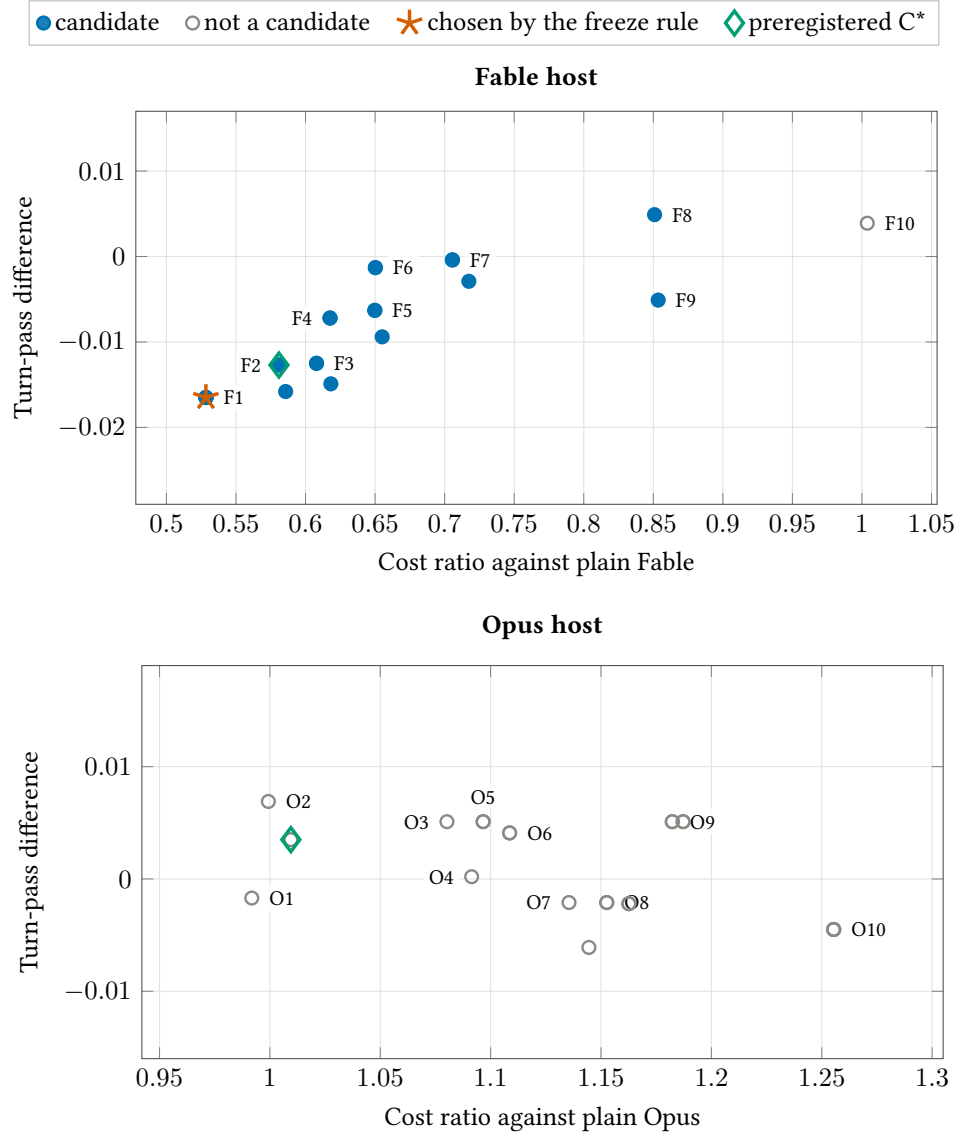


Figure 13: On Fable many configurations qualify; on Opus none does. All 23 Fable and 23 Opus configurations the freeze rule considered: cost ratio against the host’s plain default and turn-pass difference (point estimates). Keys group configurations that land on the same point; [Table 7](#) lists them.

How to read it. Left and up is better. Filled dots passed the preregistered candidate test (cost upper bound below 1, quality lower bound above the margin, final-state point no worse than the specified limit). The star is the rule’s choice; the diamond is the preregistered configuration. On Fable 22 configurations qualified; the cheapest cluster drops the scope gate. On Opus none qualified, so the shipped default stays.

6.7 Effort, across three hosts

Three separate studies measured medium against default reasoning effort, one per host model. Figure 14 puts them on one chart.

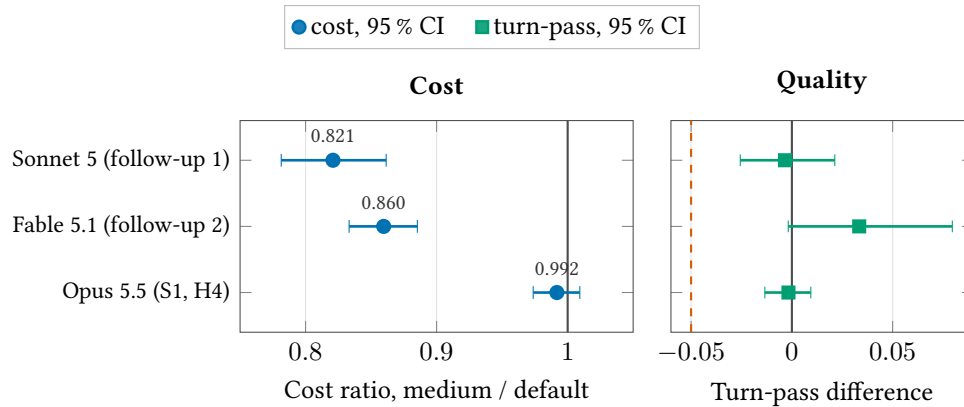


Figure 14: Medium effort saved money on Sonnet and Fable but did not demonstrate a saving on Opus. Medium against default reasoning effort, measured in three studies: Sonnet (follow-up 1), Fable (follow-up 2) and Opus (S1, H4). 95 % intervals.

How to read it. Sonnet and Fable sit well left of 1: medium effort is cheaper with no detectable quality loss. Opus sits on 1. The same setting saves money on two models and nothing on the third, so effort must be set per host.

6.8 Where the money goes

Why does the same switch save money on one host and cost money on the other? Figure 15 splits the average S1 session's cost by token class.

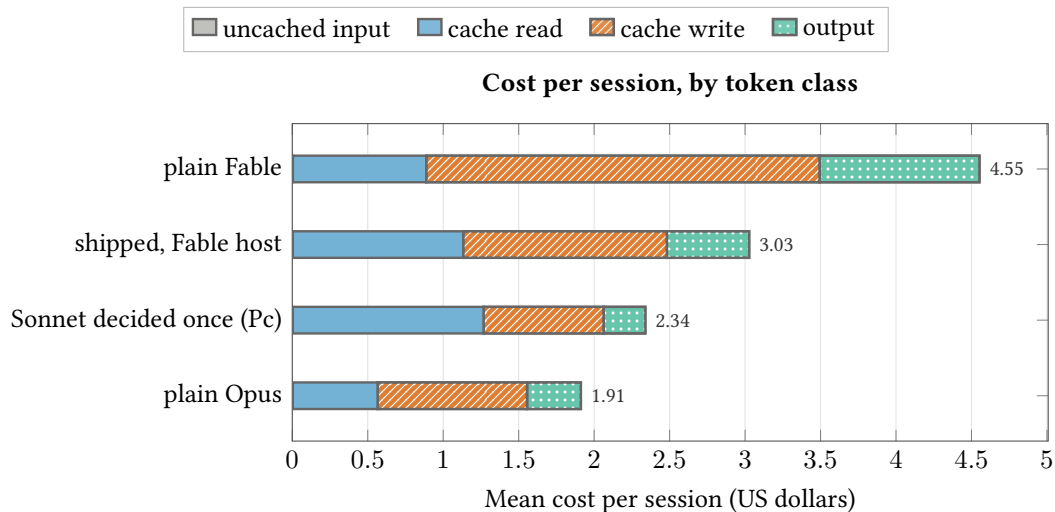


Figure 15: Plain Fable spends most of its money writing to the cache. Mean cost per S1 session split by token class, at the study's price table (exploratory).

How to read it. Each bar is the average session. Plain Fable spends most on cache writes (57 % of \$4.55): writing context into Fable's cache is expensive. The Sonnet session (\$2.34) spends more on cache reads (54%), which are cheap per token but many. Plain Opus (\$1.91) is already cheaper than the Sonnet session, which is the whole H5 story in one bar.

6.9 Does the answer replicate?

The routing result has now been measured on four independent sets of scenarios. Figure 16 lines them up per host.

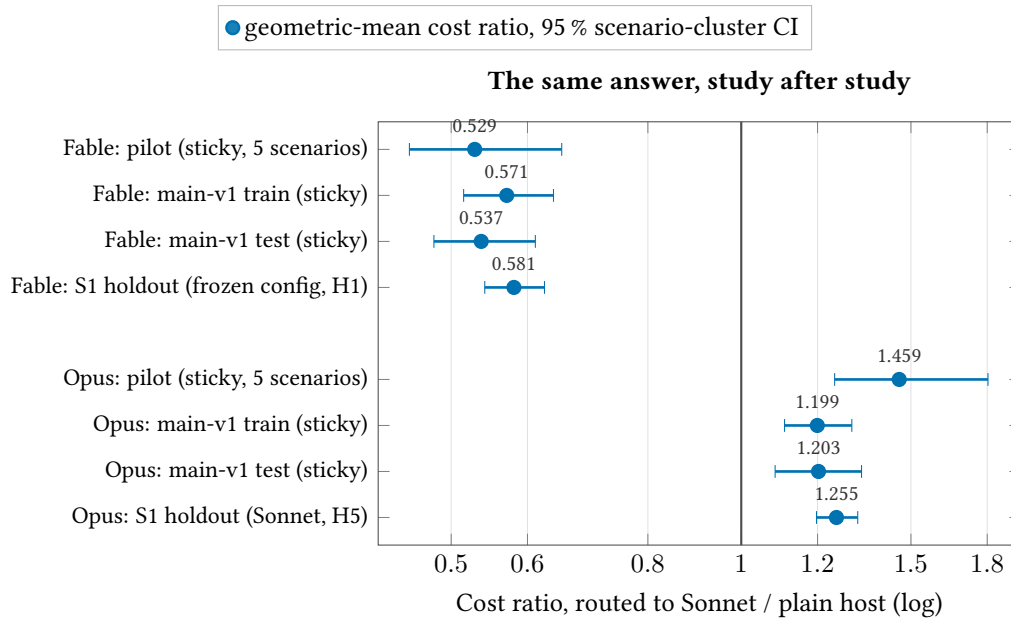


Figure 16: Four independent sets of scenarios give the same answer for each host. Routing to Sonnet against the plain host, per host, across the pilot, the main-v1 training and test splits and S1. Main-v1 rows are the sticky arm without the quality filter; the S1 rows are H1 (Fable) and H5 (Opus).

How to read it. Read top to bottom within a host. The Fable rows (0.529, 0.571, 0.537, 0.581) all lie near one half; the Opus rows (1.459, 1.199, 1.203, 1.255) all lie above 1. Four independent sets of scenarios, the same sign and roughly the same size.

What this taught us, and what it left open.. Inside S1 the headline holds up: the direction is the same in almost every scenario and in four independent scenario sets, the rule and the decision model can barely differ, and cache writes explain why Fable and Opus point in opposite directions. What stays thin is quality on review and explain work, where non-inferiority was not established, and the Fable configuration the freeze rule selected on the holdout, which must replicate before it ships. Section 7 turns this into settings.

7 What to configure on Monday

Key idea. What gets the most out of a decision model: decide once per session; let price arithmetic gate routing (Opus: never route; Fable: route to Sonnet at medium); use a deterministic rule where it matches the model, as it did for the session-start choice; set effort to medium where it helps (Sonnet, Fable, not Opus); keep review and explain work on the host. Where a decision model may still pay is in decisions whose right answer changes within a session; that is a hypothesis for S2–S4, not a finding.

Table 6 gives the settings we recommend, per host model, after S1.

Table 6: Recommended settings per host model at the companion report’s prices, after S1.

Host model	Session model	Reasoning effort	Evidence
Fable 5.1	Route to Sonnet, decided once by rule R* (scope gate at 300 files); review- and explain-like first prompts kept on the host via a keyword proxy	Sonnet at medium; Fable at medium when kept (0.860×)	expected about 0.75×, turn-pass about 0 (replay, exploratory); R* alone 0.581× (S1 H1)
Opus 5.5	Keep the host: the price gate never routes (routing 1.255×)	Default (medium 0.992×, no saving)	S1 H3, H4, H5
Sonnet 5	Already the cheap model: no routing	Medium (0.821× its default)	Sonnet follow-up

What a user should set..

- **Price gate on.** The gate compares the host’s and Sonnet’s prices with the request mix measured in main-v1 (request multipliers 1.11 for Fable and 1.38 for Opus, re-derived by a replay of the shipped code, which passed) and keeps the host when routing cannot pay.
- **Decide once.** One model and one effort per session; switching either rewrites the prompt cache.
- **Effort medium on Sonnet and Fable**, default on Opus.
- **Decider: the rule R*.** It matched Jev on the session-start choice (H2) and needs no external call. Jev stays available as an opt-in decider; it sends the session’s starting state to an external service and should run only with the user’s consent (`allow_external_state`).
- **keep_on_host: [review, explain]** on Fable, per H7: a deliberate quality-first choice. Without a judge there is no task-type label, so the shipped bundle uses a frozen keyword classifier (`intent-kw-v1`) as a proxy: it keeps sessions whose first prompt reads as a question on the host. On the 135 scenario prompts of all three panels it catches 26 of the 28 review and explain sessions, but also 43 other read-only questions (precision 38 %); it was not tuned on the holdout. Replayed on the S1 sessions (exploratory, not preregistered), against plain Fable:
 - R* alone: 0.581×, turn-pass −0.013 (the preregistered H1 result);
 - R* with review and explain kept on the host by their true labels: 0.650× (0.601–0.703), turn-pass −0.006;
 - R* with the shipped keyword proxy: 0.749× (0.699–0.799), turn-pass 0.002.

The proxy gives up part of the saving (about 0.75× instead of about 0.58×) to avoid the review and explain quality risk. Users who accept that risk can remove `keep_on_host` to get about 0.58×. A better classifier is S2 work. In the running example, `click-flow-explain` is an explain task, and the shipped proxy leaves it kept on Fable: the quality-first choice forgoes the 0.580× it showed in S1.

Per harness.. In Amplifier, the fast-decisions bundle makes the decision inside every session; install it with `amplifier bundle add` as in the repository README and keep it current with `amplifier bundle update`. In Claude Code, Codex or Copilot CLI, the smart tool makes the same decision before the harness starts. Its `decide` command returns the model and effort for a task; `launch` decides and then starts the harness with them:

```
amplifier-fast-decisions decide    amplifier-fast-decisions launch
```

Install and update the skill with:

```
npx skills add michaeljabbour/amplifier-bundle-fast-decisions
npx skills update
```

8 Configuration recipes

The recipes below use only keys that exist in the shipped bundle (version 0.3.0). The first listing is the orchestrator configuration exactly as it ships in `behaviors/fast-decisions.yaml`; the others change one or two keys of it.

The shipped default (Fable, Opus and Sonnet hosts alike).. The same configuration serves every host: the price gate keeps Opus sessions on Opus, `by_host` sets Fable's own effort to medium, and `keep_on_host` keeps review- and explain-like sessions on the host.

```
config:
  mode: active
  backend: jev
  allow_external_state: false
  read_shortcut: false
  timeout_ms: 3000
  allowed_tools: [fast_workspace]
  effort_routing:
    by_tier:
      cheap: medium
      strong: null
    by_host:
      claude-fable-5-1:
        strong: medium
  model_routing:
    start_model: claude-sonnet-5
    provider_match: anthropic
    start_policy: rules
    complex_min_prompt_chars: null
    decision_scope: session
    price_gate:
      enabled: true
    keep_on_host:
      task_types: [review, explain]
    cheap_max_workspace_files: 300
    max_requests_before_escalation: null
    escalate_on_test_failure: false
    escalate_on_provider_error: true
  cache_keepalive:
    enabled: true
    interval_s: 270
    max_refreshes: 6
    min_prefix_tokens: 20000
    max_output_tokens: 1
```

```
waste_guards:
  enabled: true
  repeat_stop: true
  retry_stop: true
  poll_wait: true
  identical_results: true
  poll_max_wait_s: 240
observatory:
  enabled: true
  open_browser: always
  port: 8765
```

Recipe: accept the review and explain risk for the larger saving (Fable).. Remove the opt-out. Expected cost falls from about $0.75\times$ to about $0.58\times$ of plain Fable, at the quality risk H7 measured.

```
model_routing:
  keep_on_host:
  task_types: []
```

Recipe: medium effort on a Sonnet host.. Sonnet at medium cost $0.821\times$ its default. The per-host map takes any model id:

```
effort_routing:
  by_host:
    claude-sonnet-5:
      strong: medium
```

Do not add Opus to this map: medium did not save money on Opus ($0.992\times$).

Recipe: use the decision model instead of the rule.. Jev chose the same session model as the rule on all but 4 of 120 scenario-reps, so this buys nothing measurable and sends the starting state to an external service. If you want it anyway, opt in explicitly:

```
allow_external_state: true
model_routing:
  start_policy: judge
```

Claude Code, Codex and Copilot CLI.. The smart tool makes the same session-start decision outside Amplifier. decide prints the choice; launch makes it and starts the harness with that model and effort (-dry-run prints the command instead):

```
npx skills add michaeljabbour/amplifier-bundle-fast-decisions
amplifier-fast-decisions decide --task "Fix the failing date parser test" \
  --host-model claude-fable-5-1 --workspace . --decider rules
amplifier-fast-decisions launch --harness claude --task "Explain the cache layer" \
  --host-model claude-fable-5-1 --workspace . --dry-run
amplifier-fast-decisions launch --harness codex --task "..." --host-model claude-fable-5-1
amplifier-fast-decisions launch --harness copilot --task "..." --host-model claude-fable-5-1
npx skills update
```

For Amplifier itself, add the bundle as in the repository README and keep it current with `amplifier bundle update`. Savings outside Amplifier have not been measured yet (S4).

9 What this means, and what remains open

9.1 What this means

We set out to learn what a fast decision model buys an agent harness. The answer has three parts. Decisions are fast and cheap, so speed is never the obstacle. On labelled decisions the best-value model is Jev, but on real read decisions no model was safe enough to act alone. And where the money is, the model a whole session runs on, the measured saving came from price arithmetic and a deterministic rule, not from the decision model: routing to Sonnet at medium effort saves on Fable ($0.581\times$, [Section 5](#)), never routing is right on Opus, and the rule matched the model at the session start.

The configuration answer. Decide once per session, keep the price gate on, use the rule R^* , set effort to medium on Sonnet and Fable but not Opus, and keep review and explain work on the host ([Section 7](#)).

9.2 What remains open

The plan for this program (`docs/design/v3/PLAN.md`) has five sessions studies; only the first is in this report. Each of the others has a written design there:

- **S2, long sessions:** does decide-once still hold when a session runs far beyond 16 turns?
- **S3, delegation in situ:** routing the sub-tasks an agent delegates, inside real sessions.
- **S4, other harnesses:** the measured saving of decide and launch in Claude Code, Codex and Copilot CLI.
- **S5, SWE-bench at full scale:** two arms on the full benchmark.
- **Replicating the Fable freeze choice in S2** (it was selected on holdout), and anything on hosts other than Fable, Opus and Sonnet.

Honest limits.. Everything here ran on one provider, at one price table: costs are list prices applied to measured tokens, tools-normalized as described in [Section 5](#), not invoices, and the Opus conclusion depends on its cache-read price. The sessions are scripted, at most 16 turns, on public code; the counterfactual reuses main-v1's sessions and its cheap outcomes for a few scenario-reps are estimates. The telemetry is one owner's workload. Decision quality on real reads remains unidentified beyond the 42 labelled cases. The judge's own cost is small but was estimated, not billed. S1 had operational deviations ([Section 5](#)): a raised budget cap, a stop rule not followed to the letter, and many infrastructure reruns.

Where a decision model might still matter. The session-start choice is made once, with little information, and a rule already captures it. A decision model may still earn its keep where the right answer changes during a session: escalating a stuck session to the host, choosing which sub-task to delegate, or stopping a loop. That is a hypothesis for S2–S4, not a finding.

The takeaway. At the start of a session, price arithmetic and a one-line rule captured all of the measured saving; a fast decision model will have to prove its worth on decisions that change during the session.

A Companion material

Companion technical report.. The full studies behind [Sections 3](#) and [4](#) (design, power, running safely, every confirmatory and exploratory result, the effort follow-ups, three rounds of review, data dictionary and reproduction) are in

[docs/papers/2026-10-02-paired-measurement/](#). The external reviews and their verification are in [docs/reviews/2026-10-paired-measurement/](#).

Evidence used here..

- Observatory and counterfactual: [docs/evidence/2026-10-v3-offline/](#) (A1, A0, A3 inputs).
- Judge benchmark, real traces, Clef:
[docs/evidence/2026-09-30-judge-benchmark/](#)
[docs/evidence/2026-10-01-trace-judge-benchmark/](#)
[docs/evidence/2026-10-04-clef-judges/](#)
- Paired campaign and follow-ups:
[docs/evidence/2026-10-02-paired-campaign/](#)
[docs/evidence/2026-10-05-effort-control/](#)
[docs/evidence/2026-10-06-effort-control-fable/](#)
[docs/evidence/2026-10-05-defaults-replay/](#)
- S1:
[docs/evidence/2026-10-06-holdout-v3/](#)
preregistration: [evals/paired/PREREGISTRATION-holdout-v3.md](#)

Reproducing this document.. make rebuilds every number from the evidence and the PDF; make check checks figure geometry and, for every bar and forest chart, that each rendered row label sits beside its own value; make FINAL=1 additionally requires the S1 result and fails without it.

B Freeze-rule configurations

Table 7: Every configuration the S1 freeze rule considered (keys as in [Fig. 13](#)). Cost ratio and turn-pass difference against the host’s plain default.

Key	Host	Configuration	cost ratio	turn-pass Δ	candidate
F1	Fable	always_route strong default scope off keep none	0.528	−0.017	✓
F1	Fable	always_route strong medium scope off keep none	0.528	−0.017	✓
F1	Fable	rstar strong default scope off keep none	0.528	−0.017	✓
F1	Fable	rstar strong medium scope off keep none	0.528	−0.017	✓
F2	Fable	always_route strong medium scope 300 keep none	0.581	−0.013	✓
F2	Fable	rstar strong medium scope 300 keep none	0.581	−0.013	✓
F2	Fable	jev strong medium scope 300 keep none	0.586	−0.016	✓
F3	Fable	always_route strong default scope 300 keep none	0.608	−0.012	✓

Key	Host	Configuration	cost ratio	turn-pass Δ	candidate
F3	Fable	rstar strong default scope 300 keep none	0.608	−0.012	✓
F3	Fable	jev strong default scope 300 keep none	0.618	−0.015	✓
F4	Fable	always_route strong medium scope off keep review+explain	0.618	−0.007	✓
F4	Fable	rstar strong medium scope off keep review+explain	0.618	−0.007	✓
F5	Fable	always_route strong medium scope 300 keep review+explain	0.650	−0.006	✓
F5	Fable	rstar strong medium scope 300 keep review+explain	0.650	−0.006	✓
F5	Fable	jev strong medium scope 300 keep review+explain	0.655	−0.009	✓
F6	Fable	always_route strong default scope off keep review+explain	0.650	−0.001	✓
F6	Fable	rstar strong default scope off keep review+explain	0.650	−0.001	✓
F7	Fable	always_route strong default scope 300 keep review+explain	0.706	0.000	✓
F7	Fable	rstar strong default scope 300 keep review+explain	0.706	0.000	✓
F7	Fable	jev strong default scope 300 keep review+explain	0.717	−0.003	✓
F8	Fable	plain-medium	0.851	0.005	✓
F9	Fable	bundle-unrouted medium	0.853	−0.005	✓
F10	Fable	bundle-unrouted default	1.004	0.004	
O1	Opus	plain-medium	0.992	−0.002	
O2	Opus	bundle-unrouted default	0.999	0.007	
O2	Opus	bundle-unrouted medium	1.009	0.003	
O3	Opus	jev strong default scope 300 keep review+explain	1.080	0.005	
O4	Opus	jev strong medium scope 300 keep review+explain	1.091	0.000	
O5	Opus	always_route strong default scope 300 keep review+explain	1.097	0.005	
O5	Opus	rstar strong default scope 300 keep review+explain	1.097	0.005	
O6	Opus	always_route strong medium scope 300 keep review+explain	1.109	0.004	
O6	Opus	rstar strong medium scope 300 keep review+explain	1.109	0.004	
O7	Opus	jev strong default scope 300 keep none	1.136	−0.002	
O7	Opus	jev strong medium scope 300 keep none	1.144	−0.006	
O8	Opus	always_route strong default scope 300 keep none	1.153	−0.002	
O8	Opus	rstar strong default scope 300 keep none	1.153	−0.002	
O8	Opus	always_route strong medium scope 300 keep none	1.163	−0.002	
O8	Opus	rstar strong medium scope 300 keep none	1.163	−0.002	
O9	Opus	always_route strong default scope off keep review+explain	1.182	0.005	

Key	Host	Configuration	cost ratio	turn-pass Δ	candidate
O9	Opus	rstar strong default scope off keep review+explain	1.182	0.005	
O9	Opus	always_route strong medium scope off keep review+explain	1.187	0.005	
O9	Opus	rstar strong medium scope off keep review+explain	1.187	0.005	
O10	Opus	always_route strong default scope off keep none	1.255	−0.004	
O10	Opus	always_route strong medium scope off keep none	1.255	−0.004	
O10	Opus	rstar strong default scope off keep none	1.255	−0.004	
O10	Opus	rstar strong medium scope off keep none	1.255	−0.004	

C Glossary

A/A pair

Two identical plain-host sessions; their difference shows the noise floor.

Cache read / cache write

Tokens served from, or written into, the provider’s prompt cache. Reads are cheap; writes cost more than ordinary input.

Confidence interval (95 %)

An interval produced by a procedure that, over many repetitions of the study, would contain the true value of the parameter 95 % of the time; here computed by resampling whole scenarios.

Decide once

Choose the session’s model and effort at the start and never switch.

Decision model (system 1)

A fast model that answers a typed question with a probability per allowed answer.

Freeze rule

The preregistered, deterministic rule that picks each host’s configuration from the S1 results.

Geometric mean

The average of logs, exponentiated; the right average for ratios.

Holm correction

A way to keep the overall false-positive rate at 5 % when testing several hypotheses.

Host model

The large model the agent normally runs on (Fable 5.1 or Opus 5.5 here).

keep_on_host

A setting that keeps chosen task types (review, explain) on the host model.

Non-inferiority margin

How much worse quality may be and still count as “no worse than the specified margin”: 5 points of turn-pass.

Paired cost ratio

One arm’s session cost divided by its partner’s, on the same task, started together.

Preregistered

Hypotheses and thresholds fixed in a committed document before the data existed.

Price gate

Arithmetic that keeps the host when the cheaper model cannot save money at current prices.

Rule R*

Route to Sonnet unless the workspace has more than 300 files.

Scope gate

The workspace-size part of R*.

Turn-pass fraction

The share of a session’s scripted turns whose checks passed.

D Response to the independent review (round 3)

An independent reviewer read the 25-page version of this report, reran the S1, main-v1 and effort analyses from the published rows (all reproduced), and recommended publication after focused corrections. The review is kept with the review record:

`docs/reviews/2026-10-paired-measurement/round3-independent-review-v3.md`

Each item and the change:

1. **Row labels reversed in four figures** (event mix, S1 hypotheses, effort across hosts, cost composition). Fixed: those charts’ data rows are now written in the order pgfplots assigns tick labels. Every bar and forest chart now prints each row’s value beside it, and a new build-failing check reads every rendered row label and the value next to it from the PDF and compares them with the source table ([Section A](#)). The check reports the original defect when the old row order is restored.
2. **Statistical wording.** H7 now reads “non-inferiority not established”; judge accuracy reads “no accuracy difference was detected after the preregistered correction” (not equivalence; H2 remains a true equivalence test); H3 reads “cost-equivalent within the preregistered $\pm 5\%$ margin”; Opus effort reads “medium effort did not demonstrate a saving”.
3. **Holdout and observatory descriptions.** The S1 panel is described as disjoint from main-v1 and smoke-tested before preregistration, with the pre-freeze repair account; the observatory window distinguishes all events (to the snapshot cutoff) from the first week.
4. **S1 methods in the main text.** A methods box defines quality and its limits, the cost basis and the composed policies, using the reviewer’s wording for the latter, with H6 as the empirical

check of the cost reconstruction. H7 and the subgroup figure distinguish cost and quality denominators.

5. **Effort scale.** The cross-study log-cost share is replaced by “medium effort independently reduced Fable cost by about 14 % in the follow-up; routing remained the larger cost lever.”
6. **Measurement and rebuild.** The tools normalization is defined and sized (0.45 % in main-v1, 0.135 % in S1). Every input read from git is pinned to a recorded commit instead of a moving branch.
7. **Related work.** A paragraph places the report next to RouteLLM and FrugalGPT.
8. **Editorial.** The confidence-interval definition now states long-run coverage; non-inferiority reads “no worse than the specified margin”; very small p-values print as < 0.001 ; key-idea boxes no longer split across pages; S1 figures stay with their subsections.
9. **Author block.** David Koleczek’s title now sits under his name in the author block and nowhere else.

Alongside the review response, the post-hoc OpenAI Decisions API results were added to the judge section ([Section 3.1](#)): a table, a figure covered by the row-identity check, and a key idea stating what they do and do not change.